

The Feature Ledger

● VERSION 2 · DRAFT, NOT YET PRESENTED

Everything feature-ledger does today, in plain language. Each entry is one capability you could ask for by name, sized so the biggest pieces stand out.

What's changed since the last edition (September 8, 2026): every entry below is highlighted to show what's different. A green stripe marks a brand-new feature; a blue stripe marks an existing feature that was materially reworked, with its own **What changed** note. Anything without a stripe hasn't changed.

New feature

Updated feature

THE LEDGER AT A GLANCE

24 features across 4 areas

EVERY ONE OF THOSE 24, BY SIZE



- BIG

A capability the product is chosen for

MEDIUM

A capability in its own right, inside a bigger one

SMALL

A capability narrow enough to describe in a line

SINCE THE LAST EDITION, SEPTEMBER 8, 2026

4 NEW 6 UPDATED 14 UNCHANGED

CONTENTS

01	Setting up a ledger	6	03	Reading the ledger	3
02	Recording what the product does	10	04	The documents it produces	5
—	No longer available as of this edition	2	—	Also since last time	3

01 Setting up a ledger

6 FEATURES · 2 UPDATED

 BIG

UPDATED

Creating a ledger in an existing project

previously “One-command setup in any codebase”

The command `ledger init` creates a `.ledger` directory holding the whole corpus: the product name, the accent colour and the style guide you chose, and version 1 open as the release in progress. Everything the tool needs sits in that one directory, so a Python, Go or Rails repo looks identical to it. Setup also settles where the generated documents go. The Markdown docs are build output and are added to `.gitignore`. The PDF editions for the client are build output too, unless you run `ledger init --commit-pdfs`, which keeps them in `docs/client` so every document handed over stays in the repo.

WHAT CHANGED

- The client PDFs were written inside the gitignored output directory, with no way to keep them. A team that hands a printed edition to a client at each release wants the repo to hold what was handed over, so where the editions land is now a choice made once at setup.

 BIG

The tone every entry is written in

A ledger is written in one tone, chosen from four style guides: Google developer documentation, GOV.UK, Federal Plain Language, and ASD-STE100 Simplified Technical English. The command `ledger style` prints that tone as a sample paragraph of product prose and three feature descriptions written in it, which is the thing a writer or an agent matches. Running `ledger style set` moves the project to another guide, or to a tone you write as prose yourself in `.ledger/STYLE.md`.

 MEDIUM

Wiring into a coding agent

Setup appends a short stanza to whichever agent instruction files the repo already has — CLAUDE.md, AGENTS.md, GEMINI.md, .cursor/rules, .github/copilot-instructions.md or .windsurfrules — and creates AGENTS.md when there are none. The stanza is a pointer rather than a copy: it names six commands, the tone the project writes in, the line between recording a change and rewording an entry, and the rule that a feature change is not finished until the ledger records it. The --agents flag chooses the files, and changing the tone rewrites the stanza in every one of them.

 MEDIUM

UPDATED

Baseline survey of an existing codebase

The command ledger bootstrap prints a brief you paste into a coding agent, which then reads the codebase and records what it already does as version 1. The brief carries the entry bar, the project's tone, and five steps: list the capabilities, decide the areas, write the entries, cut the baseline from the date of the first commit, and hand back the entries it was least sure of. Because reading the whole codebase against the record is exactly what an audit does, the brief also records the survey as a full sweep, which is what gives the release gate something to measure against from the first day. The agent runs it start to finish without stopping to ask you anything.

WHAT CHANGED

- The survey ended at the baseline edition, so a freshly bootstrapped project reported its codebase as never checked and the release gate stayed quiet until somebody ran an audit by hand. The survey had in fact just swept everything, so the brief now records that before cutting — the one moment when what the ledger holds and what the codebase does are known to agree. Pass --no-audit for a survey that is deliberately partial.

 SMALL

Moving an existing ledger to a new tone

The command ledger style rewrite prints a brief for rewriting a ledger that is already written into the tone the project has moved to. The agent works one area at a time and replaces the words in place, so nothing lands on the release and no entry is marked as changed.

The authoring rules

The command `ledger rules` prints the full rules for writing the ledger: what each field means, where the bar for an entry sits, how to record a change or a removal, and which parts of the corpus you edit by hand instead. The project's tone is printed inside it, and a `RULES.md` in the ledger directory replaces the packaged text with your own house conventions.

02 Recording what the product does

10 FEATURES · 4 NEW · 1 UPDATED

Recording a capability

The command `ledger add` records one capability, with the audience it matters to, its size, its name and a present-tense description of what the product does. The id you give it is lower-case kebab-case and never changes, so it carries the feature's history through every later rename. An entry filed under an area or a sub-section that doesn't exist is refused as you write it, and the message lists the ones that do.

Recording a change to a feature

The command `ledger update` replaces a feature's description with its full current-state text, and takes a list of change bullets alongside it. Each bullet says what the feature was in the last edition and why it moved, which is the part a reader cannot get from the description. A feature that existed before this release cannot be updated without one, so an edition never carries a changed entry with no reason attached. A feature that is new in this release takes no change bullets, because the client has never seen an earlier version of it. When a feature changes a second time in one release, the command reminds you that its bullets have to be true of the last edition, not of the state your previous update left.

WHAT CHANGED

- A change bullet could be added to a feature that was new in the same release, and a second change in one release got no warning. Each bullet then described the state just before its own update, which the client never saw, so an edition could explain a fix to something that was new in the same release. The command now refuses bullets on a new feature and reminds you which edition a second change is measured against.

 BIG

NEW

Editing the ledger by hand

The corpus is JSON in the repository, one small file per feature, and the prose in it is meant to be opened and rewritten. You change how an entry is worded, add the sentence a coding agent had no way of knowing, or cut a detail the client is better off not reading — a name, a description, a dev note or the text of a change bullet. Nothing is recorded against a release and nothing is marked as changed, because the product did not move: the improved wording reads as though it had always been there, in the editions already printed as well as the next one. The command `ledger status` revalidates every file afterwards, so a description that went missing comes back as a message rather than as a broken build. What the product did keeps going through the CLI, which is what resolves the release an entry lands on and refuses a change that never says why it happened.

 BIG

NEW

The ledger audit

The command `ledger audit` reads the codebase for capabilities the ledger never recorded, which is what happens whenever someone ships without a coding agent that read the project's instructions. It prints a brief for a coding agent rather than guessing: the range since the last sweep, the changed paths already grouped into batches, and the commit subjects behind them. A sweep covers everything since the last audit, or the whole codebase with `--full`, which is the only way to find a capability the very first survey missed. The agent records what it finds and then stamps the commit it audited, so from that point the tool knows how much of the repository has never been examined. Cutting a release with unswept work behind it is refused, and says how many commits and files are outstanding. A project that started with `ledger bootstrap` is already stamped, because the baseline survey is itself a full sweep.

 BIG

NEW

Reviewing a release before it goes out

The command `ledger review` prints a brief for a coding agent to read the open release the way the client will: as one document, against the edition they already have. The brief lists what arrived in the release, every client-facing entry in it by area, and the places the drafts probably show, such as change bullets that name something the previous edition never mentioned. The agent tests every bullet against the previous edition, drops or merges the ones whose "before" never shipped, removes change notes from new entries, and makes names, numbers and cross-references agree. It edits wording only and never invents a reason, and it ends with a list of questions for you. `ledger review complete` records what was reviewed, and `ledger status` reports any client-facing entry that changed after that.

 MEDIUM

Grouping features into areas

Areas group the entries, and the order of the list is the reading order of every document the ledger produces. The command `ledger categories add` places one, before or after any other; `rename` and `remove` move the structure without touching the entries underneath it. A ledger with no areas puts everything in one called Features and prints it without a heading, so a small product reads as a list of what it does rather than as a chapter of one.

 MEDIUM

NEW

Redrafting the ledger

The command `ledger redraft start` sets the corpus aside in `.ledger/archive/` and opens the next version on an empty one, so you can write the ledger again from a fresh survey. The version numbers carry on, and every edition already printed keeps its number and its file. `ledger build --version` reprints an archived edition from its archive. The command `ledger redraft check` prints a brief for a coding agent to read every archived entry against the new corpus, put back anything the survey missed, and write a note for the client on how the arrangement changed. Until `ledger redraft complete` records that check, `ledger build` warns and `ledger release cut` refuses, and `ledger redraft on` says what is left to do. The first edition after a redraft opens with a note that it is reorganized, and marks nothing as new or updated. A redraft you did by hand, without `ledger redraft`, gets the same note when you set "redraft": true on its release in `releases.json`. That edition reuses the number of the edition before it, and its file name ends in `_redraft`, so Version 3 is followed by Version 3 (redraft) and then Version 4. On the first release it tells the client that the list replaces reports they received some other way.

 SMALL

Retiring a feature

The command `ledger remove` marks a feature as gone from this release onward, with the reason you give it. It appears once, under the heading `No longer available as of this edition`, and every earlier edition still shows the feature as it stood at the time.

 SMALL

Recording a product-wide change

The command `ledger other-change` records something that runs across the whole product, such as a wording pass over many screens or a change in what a colour means. It prints once at the end of the edition, under the heading `Also since last time`, instead of forcing the same note onto a dozen unrelated entries.

 SMALL

Sub-sections inside an area

A sub-section is a heading between an area and a feature, for an area with a real part inside it. Each one carries a name and a one-or-two-sentence overview of that part, and you file a feature into it when you record or update the feature. When every feature in a sub-section arrives in the same release, the edition flags the sub-section once rather than printing a column of identical `New` marks.

03 Reading the ledger

3 FEATURES

 MEDIUM

Listing and reading entries

The command `ledger list` prints the whole corpus, one short line per feature: its size, its id, its name, and whether it is new or changed in the release in progress. It filters by area, audience, size, or by what has moved this cycle. The command `ledger show` prints the single entry you are about to edit, and prints every version it has been through when you ask for its history.

 MEDIUM

Checking the ledger is sound

The command `ledger status` answers three questions at once: what the open release has collected so far, what the corpus holds, and how the setup resolves — the derived palette, the masthead, the fonts, the tone in force, and the browser that will print. Anything that is valid but would still print badly comes out as a warning at the end, such as an entry with no size or an area with nothing in it. It exits with an error when the corpus is broken and stays silent when there is nothing to say, so a commit hook can run it on every commit.

 SMALL

Exporting the whole ledger

The command `ledger export` prints the entire corpus as a single JSON document: the release timeline, the areas, the sub-sections, the product-wide changes, and every feature with its full history. It writes to a file instead when you name one, for a backup, a migration, or handing the data to something that isn't this tool.

■ BIG

UPDATED

Cutting a release

The command ledger release cut gives the release in progress its name and date and opens the next one behind it, which is where everything recorded from then on lands. It also records the commit the cut happened at, so an edition says which state of the product it was printed from rather than only which day, and two demos in one day stay two tellable-apart editions. The release just cut becomes an edition that never changes again, and ledger release list shows the timeline with each release's commit and the number of entries it covers. A release cut before commits were recorded gets one with ledger release amend. Cutting a release with nothing recorded against it is refused, as is one that would stand for the same day and the same commit as the release before it. So is one whose client-facing entries changed since the release was last reviewed with ledger review; the refusal names those entries.

WHAT CHANGED

- A release was a name and a date, and the date was doing work it could not do: two demos in one day are ordinary, and nothing said which of the two editions a reader was holding or which state of the code it came from. Every cut now records its commit. Sharing a date is still fine — sharing a date and a commit is refused, because at that point the two editions stand for the same moment, and --force covers a deliberate re-issue.
- A release could be cut as soon as it had something recorded against it, and nothing checked that its entries read correctly as one release. Entries are written one update at a time, so a feature that changed twice in a cycle could reach the client with a "before" they never saw, such as a fix to something that was itself new in the same edition. A cut now needs a review of the release first, unless it is the first edition or --force is passed.

The printed edition for the client

The command ledger build prints a PDF of one release: a front page carrying the masthead, the feature count, a bar breaking those features down by size, and a note of what changed since the last edition, then a contents list that links into the document, then every entry in reading order. The document is real HTML and CSS, printed through the Chrome or Edge already on the machine, with the two fonts embedded so text copies cleanly out of the finished file. An edition can print a different version number from the one the ledger counts by: set `display_version` on a release in `releases.json`, and that edition and every later one are numbered on from it. Any edition can be reprinted at any time, and it reproduces exactly what was presented that day.

WHAT CHANGED

- The contents list was a printed index: it named each area and how many features were in it, but reaching a section meant scrolling for it. Every line is now a link to the section it names, and the closing sections on what was withdrawn and what else changed are listed there too, because the edition is usually read on screen rather than on paper.
- Every edition printed the ledger's own version number, starting from 1, so a project whose client already held reports sent another way before the ledger started handed them a second Version 1. A release can now set the number its edition prints, and later editions carry on from it; the ledger's own numbering is unchanged.

Three Markdown documents for the team

The command ledger build also writes three Markdown documents from the same corpus: `FEATURES.md`, the client-facing list; `FEATURES_EXTENDED.md`, the same list with implementation notes under each entry; and `DEV_FEATURES.md`, the entries only the team sees. All three describe the latest release, so they read as what the product does today. They are build output: the corpus is what you commit, and it carries the documentation's history on its own.

 MEDIUM

UPDATED

New and changed highlighting

previously "What moved since the last edition"

Every edition marks its own changes against the edition before it: a green stripe for a feature that is new, a blue stripe and a "What changed" note for one that was reworked, a grey stripe for one that was already part of the product and is only now listed, and no stripe for an entry that stands as it did. The front page counts how many of each. A first edition has nothing behind it to compare against, so it prints as a plain inventory with no stripes anywhere on it.

WHAT CHANGED

- There were two stripes, and an entry an audit turned up had to print as one of them. A capability that had been in the product for two years then read as this sprint's work, which is the one thing a client would take issue with. The third stripe says what is actually true: the product did not move this cycle, the record did. It is a neutral grey rather than a fourth hue because green and blue are already spoken for, an accent in that band is refused for the same reason, and a fifth hue would leave a brand only reds, oranges, purples and browns to choose from.
- Whether an entry printed as New or Changed depended on whether it had a change list. A new feature that picked up a change bullet during the release printed as Changed, with a What changed note about a version the client never saw. It is now New whenever the feature first appears in that release, and a new entry never prints a change note.

 MEDIUM

Branding the documents for a client

One file, `.ledger/brand.json`, is the swap surface for a new client: a logo, one accent colour, and the two font families. The other four tints of the accent are derived from the hex you give it, and a ledger with no logo sets the product name as a serif wordmark on the front page. The accent cannot be green, blue or teal, because those hues already mean new and reworked in every edition, and ledger status warns when a brand picks one that collides.

— No longer available as of this edition

Rewording an entry — Rewording an entry is no longer a command. The words of an entry are edited in `.ledger/features/<id>.json` directly, which records nothing against a release and marks nothing as changed — the same result the command produced, without a CLI between a person and a sentence they want to fix.

Renaming a feature — Renaming a feature is no longer a command of its own. A name that was chosen badly is wording, and is edited in `.ledger/features/<id>.json`; a name the product actually changed is recorded with `ledger update`, which carries the reason the reader is given. Every document still footnotes the previous name.

Also since last time

Changes that run across the whole product rather than belonging to any one feature above.

- The tool is published on npm as `feature-ledger`. You install it once with ``npm install -g feature-ledger`` and run ``ledger`` in any project, add it to a JavaScript project with ``npm install --save-dev feature-ledger``, or run any command through ``npx feature-ledger`` without installing anything. Using it no longer means keeping a checkout of the tool itself.
- Every command that prints a prompt for a coding agent (`ledger bootstrap`, `ledger style rewrite`, `ledger audit`, `ledger review` and `ledger redraft check`) takes `--prompt-only`, which prints the prompt alone. You can pipe it straight to the clipboard, for example with `ledger review --prompt-only | pbcopy`. Warnings from the explanation still print on your screen.
- Adding `--help` or `-h` to any command prints that command's part of the help and runs nothing. For example, `ledger release cut --help` never cuts a release.

`feature-ledger` · The Feature Ledger · Version 2. Prepared for the client; every entry describes behaviour that is built and running.